Table 11-21. *CSS combinators*

Combinator	Detail	Example
" "	Descendant at any depth	`main div p h1.large #special a`
>	Immediate child	`div > p`
~	Subsequent siblings: `A ~ B` matches *all* B preceded by A where both share a parent	`h1 ~ div`
+	Next sibling: `A + B` matches *first* B preceded by A	`h1 + p`

Descendant and immediate child have obvious parallels in the XPath `child::` and `descendant::` axes. Subsequent ~ behaves like `following-sibling::` and next sibling + behaves like `following-sibling::node[1]`. There also exists the "column combinator" || (which is misnamed) and the "namespace separator/combinator" | (it is also not a combinator) neither of which you are likely to need to worry about.

Hopefully the above has given some insight into how powerful and flexible CSS selector expressions can be, and I think it's time we saw some practical PHP examples that will give XPath a run for its money:

```
// ... $html5 heredoc as before
$dom = Dom\HTMLDocument::createFromString($html5);
$parasAfterLarge = $dom->querySelectorAll('main > p.large ~ p');
print "Selected paragraphs:" . PHP_EOL;
foreach ($parasAfterLarge as $node) {
    print $node->textContent . PHP_EOL;
}

$firstParaAfterLarge = $dom->querySelectorAll('main > p.large + p');
print "Selected paragraphs:" . PHP_EOL;
foreach ($firstParaAfterLarge as $node) {
    print $node->textContent . PHP_EOL;
}

Output:
Selected paragraphs:
Para Two
```